

UNIT III

Trees: Binary tree, Terminology, Representation, Traversals, Applications - Binary search tree - AVL tree. B Trees: B Tree indexing, operations on a B Tree, Lower and upper bounds of a B Tree - B + Tree Indexing - Trie Tree Indexing

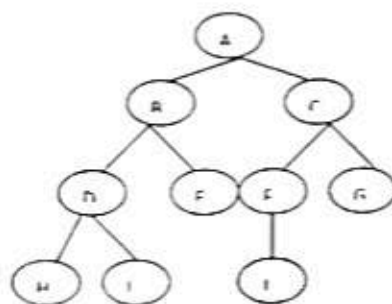
TREES

Basic terminologies

TREE: is defined as a finite set of one or more nodes such that

- there is one specially designated node called **ROOT** and
- the remaining nodes are partitioned into a collection of sub-trees of the root each of which is also a tree.

Example



LEVEL

0

1

2

3

NODE: Node of a tree stores the actual data and links to the other node.

The nodes of a tree have a parent-child relationship. The root does not have a parent ; but each one of the other nodes has a parent node associated to it . A node may or may not have children is called a **leaf node** or **terminal nodes**.

A line from a parent to a child node is called a **branch** . If a tree has n nodes, one of which is the root there would be $n-1$ branches.

The number of sub-trees of a node is called its degree. The degree of A is 2, F is 1 and J is zero. The leaf node is having degree zero and other nodes are referred as non-terminals.

The degree of a tree is the maximum degree of the nodes in the tree.

Nodes with the same parent are called siblings. Here D & E are all siblings. H & I are also siblings.

The level of a node is defined by initially letting the root be at level zero. If a node is at level l , then its children are at level $l+1$.

The height or depth of a tree is defined to be the maximum level of any node in the tree.

A set of trees is called forest; if we remove the root of a tree we get a forest. In the above fig, if we remove A, we get a forest with three trees.

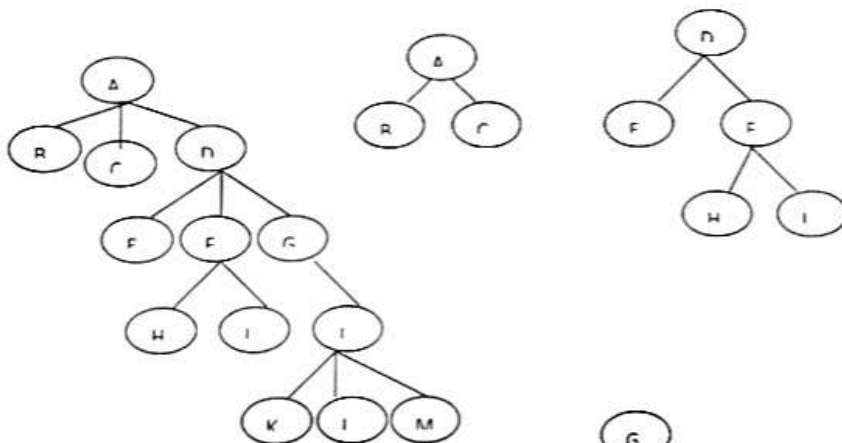


FIG: Tree

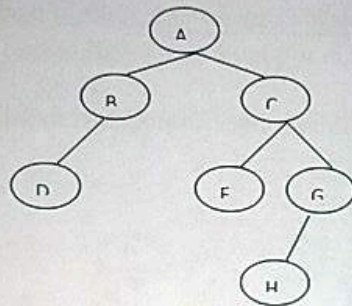
FIG: Forest(Sub-trees)

PROPERTIES OF A TREE:

1. Any node can be the root of the tree and each node in a tree has the property that there is exactly one path connecting that node with every other node in the tree. The tree in which the root is identified is called a rooted tree; a tree in which the root is not identified is called a free tree.
2. Each node, except the root, has a unique parent.

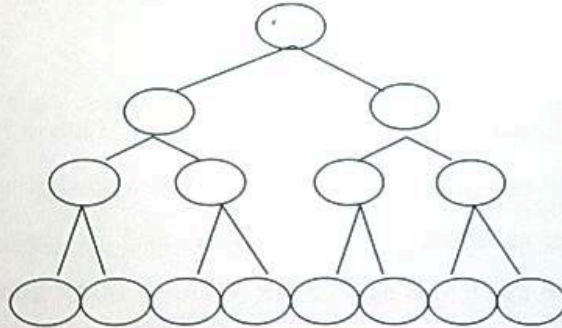
BINARY TREES:

A binary tree is a tree, which is, either empty or consists of a root node and two disjoint binary trees called the left sub-tree and right sub-tree.

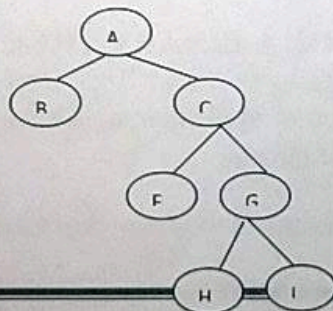
**FIG: Binary Tree**

In a binary tree, no node can have more than two children. So every binary tree is a tree, not every tree is a binary tree.

A **complete binary tree** is a binary tree in which all internal nodes have degree and all leaves are at the same level

**FIG: Complete binary tree**

If every non-leaf node in a binary tree has non-empty left and right sub-trees, the tree is termed as **strictly binary tree**

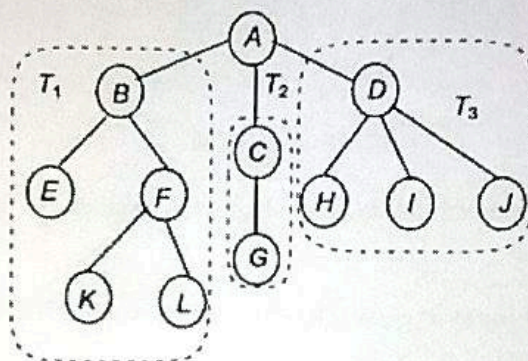
**FIG: Strictly binary tree**

If A is the root of a binary tree and B is the root of its left or right sub-tree, then A is said to be the parent of B and B is said to be the left or right child of A.

Node n_1 is an **ancestor** of node n_2 , if n_1 is either the parent of n_2 or the parent of some ancestor of n_2 .

Here n_2 is a **descendant** of n_1 , a node n_2 is a **left descendant** of node n_1 if n_2 is either the left child of n_1 or a descendant of the left child of n_1 .

A **right descendant** may be similarly defined the number of nodes at level i is 2^i . For a complete binary tree with k levels contains **2^k nodes**.



A sample tree T .

- To illustrate the above definition, consider the sample tree as given in fig.
- In the sample tree T , there is a set of 12 nodes. Here A is a special node being the root of the tree. Remaining nodes are partitioned into 3 sets T_1, T_2 and T_3 ; they are sub trees of the root node A . by definition, each sub tree is again a tree. Observe that a tree is defined recursively. A same tree can be expressed in a string notation as shown below.

BINARY TREE REPRESENTATION:

LINEAR REPRESENTATION OF A BINARY TREE

The linear representation method of implementing a binary tree uses a one-dimensional array of size $((2^{d+1})-1)$ where d is the depth of the tree.

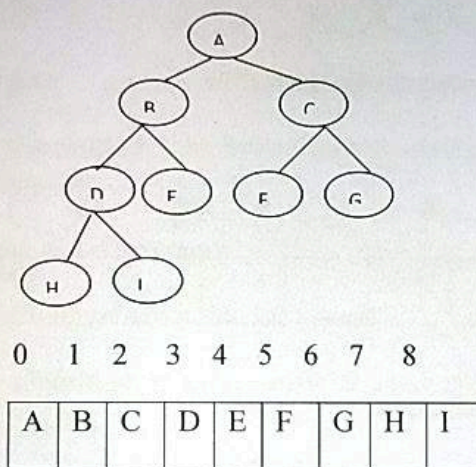
Once the size of the array has been determined the following method is used to represent the tree.

1. Store the root in 1st location of the array.

2. If a node is in location n of the array store its left child at location $2n$ and its right child at $(2n+1)$.

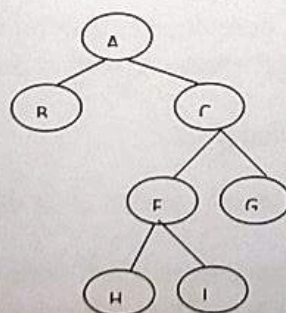
In c, arrays start at position 0; therefore instead of numbering the trees nodes from 1 to n , we number them from 0 to $n-1$. The two child of a node at position P are in positions $2P+1$ and $2P+2$.

The following figure illustrate arrays that represent the almost complete binary trees.

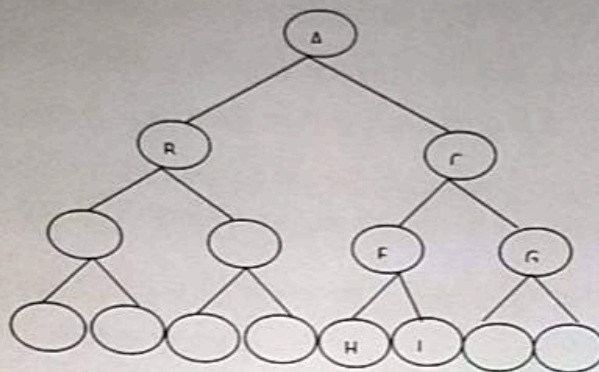


We can extend this array representation of almost complete binary trees to an array representation of binary trees generally.

The following fig(A) illustrates binary tree and fig(B) illustrates the almost complete binary tree of fig(a). Finally fig(C) illustrates the array implementation of the almost complete binary tree.



Fig(A) Binary tree



Fig(B) Almost a complete binary tree

0	1	2	3	4	5	6	7	8	9	10	11	12
A	B	C			F	G					H	I

Fig (C) Array Representation

ADVANTAGES:

1. Given a child node , its parent node can be determined immediately. If a child node is at location N in the array, then its parent is at location $N/2$.
2. It can be implemented easily in languages in which only static memory allocation is directly available.
- 3.

DISADVANTAGES:

1. Insertion or deletion of a node causes considerable data movement up and down the array, using an excessive amount of processing time.
2. Wastage of memory due to partially filled trees.

LINKED LIST REPRESENTATION:

Linked lists most commonly represent binary trees. Each node can be considered as having 3 elementary fields : a data field, left pointer, pointing to left sub-tree and right pointer pointing to the right sub-tree.

The following figure is an example of linked storage representation of a binary tree.

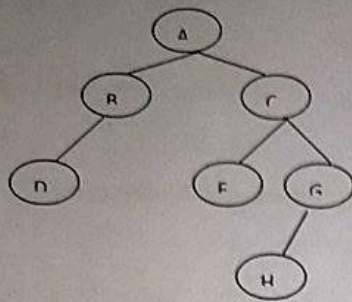


FIG: Binary tree

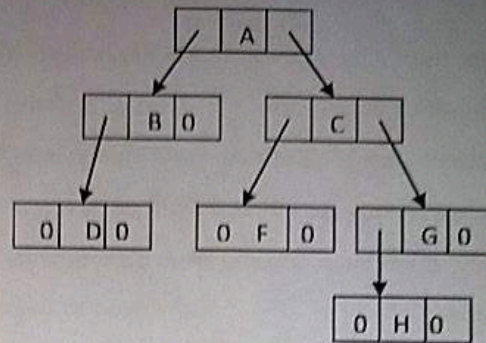


FIG: Linked representation of a binary tree

Although for most purposes the linked representation of a binary tree is efficient, it does have certain disadvantages. Namely,

1. Wasted memory space is well pointers.
2. Given a node, it is difficult to determined to parent.
3. Its implementation algorithm is more difficult in languages that do not offer dynamic storage techniques.

BINARY TREE TRAVERSALS (***)

Explain all binary tree traversal procedure with neat diagrams. (11 Marks Nov 2013, Nov 2014, Nov 2015)

Another common operation is to traverse a binary tree; that is, to pass through the tree, enumerating each of its nodes once.

- The contents of each node can be printed or processed. In either case, each node is visited as it enumerated.
- The order in which the nodes of a linear list are visited in a traversal is clearly from first to last.
- However, there is a no such "natural" linear order for the nodes of a tree. Thus, different ordering is used for traversal in different cases.

- There are three traversal methods. In each of these methods, nothing need to done to traverse an empty binary tree.
- The methods are all defined recursively, so that traversing a binary tree involves visiting the root and traversing its left and right subtrees.
- The only difference among the methods is the order in which these three operations are performed.

1. **To traverse a nonempty binary tree in preorder** (also known as depth-first order), we perform the following three operations;

1. Visit the root.
2. Traverse the left subtree in preorder.
3. Traverse the right subtree in preorder.

```
void pretrav(NODEPTR tree)
```

```
{
    if (tree!=NULL)
    {
        printf("%d", tree->info);
        pretrav(tree->left);
        pretrav(tree->right);
    }
}
```

2. **To traverse a nonempty binary tree inorder** (or symmetric);

1. Traverse the left subtree in inorder.
2. Visit the root.
3. Traverse the right subtree in inorder.

```
void intrav(NPDEPTR tree)
```

```
{
    if(tree!=NULL)
    {
        intrav(tree->left);
```



```

        printf("%d\n", tree->info);
        intrav(tree->right);
    }
}

```

3. To traverse a nonempty binary tree in postorder;

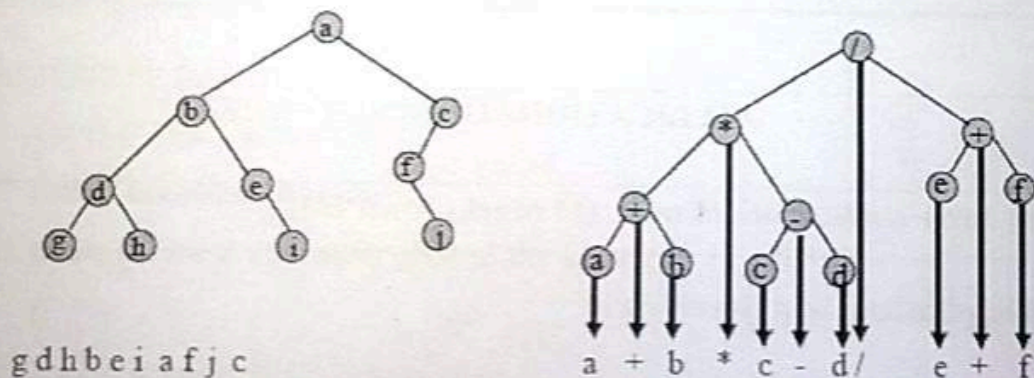
1. Traverse the left subtree in postorder.
2. Traverse the right subtree in postorder.
3. Visit the root.

```

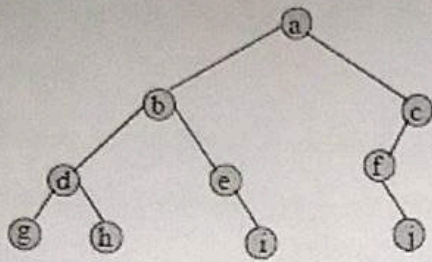
void posttrav(NODEPTR tree)
{
    if(tree!=NULL)
    {
        posttrav(tree->left);
        posttrav(tree->right);
        printf("%d\n", tree->info);
    }
}

```

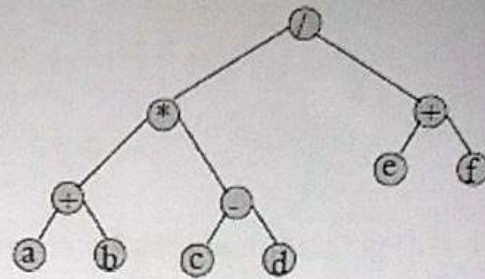
Figure illustrates two binary trees and their traversal in preorder, inorder and postorder.



Inorder traversal

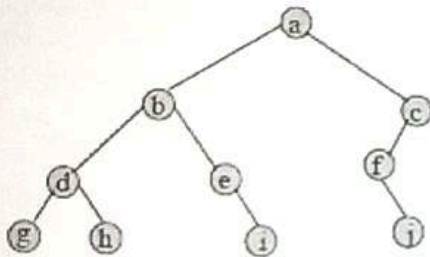


a b d g h e i c f j

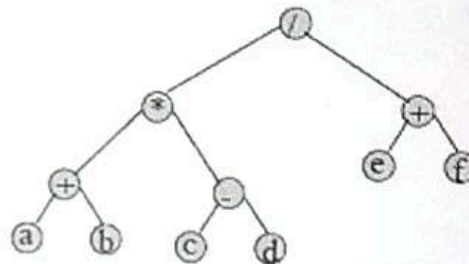


/ * + a b - c d + e f

Preorder traversal



g h d i e b j f c a



a b + c d - * e f + /

Postorder Traversal

APPLICATIONS OF BINARY TREE

Describe any three applications of trees. (11 Marks April 2010)

There are 4 applications of a Binary Tree

- Finding out the Duplicate
- Traversal
- Sorting
- Evaluation of Expression Trees

1. Finding out the Duplicate

- A binary tree is a useful data structure when two-way decisions must be made at each point in a process.
- For example, suppose that it is necessary to find all duplications in a list of numbers. One way of doing this is to compare each number with all those that precede it. However, this involves a large number of comparisons.
- The numbers of comparisons can be reduced by using a binary tree.
- The first number in the list is placed in a node that is established as the root of a binary tree with empty left and right subtrees.
- Each successive number in the list is then compared to the number in the root. If it matches, there is a duplicate.
- If it is smaller, examine the left subtree; if it is larger, examine the right subtree.
- If the subtree is empty, the number is not a duplicate and is placed into a new node at that position in the tree.
- If the subtree is nonempty, compare the number to the contents of the root of the subtree and the entire process is repeated with the subtree.
- An algorithm for doing this follows.

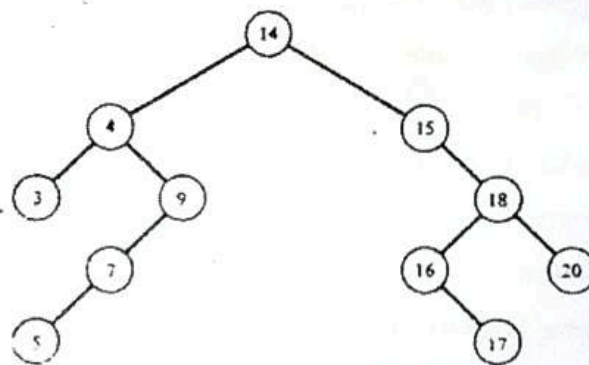
```
scanf ("%d", &number);
tree = maketree(number);
while (there are numbers left in the input)
{
    scanf ("%d", &number);
    p=q=tree;
    while (number!=info(p) && q!=NULL)
    {
        p=q;
        if (number<info(p))
```



```

        q == left(p);
        else
        q = right(p);
    }
    if (number==info(p))
        printf ("%d %s \n", number, "is a duplicate");
    else if (number<info(p))
        setleft (p, number);
    else
        setright (p, number);
}

```



A binary tree constructed for finding duplicates.

Binary tree constructed for finding duplicates

3. Traversal

Explain the traversals of binary tree with examples. (11 Marks April 2014)

Explain all binary tree traversal procedures with neat diagrams. (11 Marks Nov 2013)

What are the three tree traversal methods in a tree? (11 Marks April 2015)

- Another common operation is to traverse a binary tree; that is, to pass through the tree, enumerating each of its nodes once.
- The contents of each node can be printed or processed. In either case, each node is visited as it is enumerated.

- The order in which the nodes of a linear list are visited in a traversal is clearly from first to last.
- However, there is no such "natural" linear order for the nodes of a tree. Thus, different ordering is used for traversal in different cases.
- There are three traversal methods. In each of these methods, nothing need to done to traverse an empty binary tree.
- The methods are all defined recursively, so that traversing a binary tree involves visiting the root and traversing its left and right subtrees.
- The only difference among the methods is the order in which these three operations are performed.
- To traverse a nonempty binary tree in preorder (also known as depth-first order), we perform the following three operations;
 1. Visit the root.
 2. Traverse the left subtree in preorder.
 3. Traverse the right subtree in preorder.

```
void pretrav(NODEPTR tree)
{
    if (tree!=NULL)
    {
        printf("%d", tree->info);
        pretrav(tree->left);
        pretrav(tree->right);
    }
}
```

To traverse a nonempty binary tree inorder (or symmetric);

4. Traverse the left subtree in inorder.
5. Visit the root.
6. Traverse the right subtree in inorder.

```
void intrav(NPDEPTR tree)
{
```

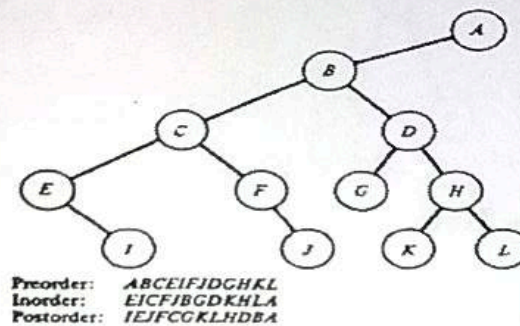
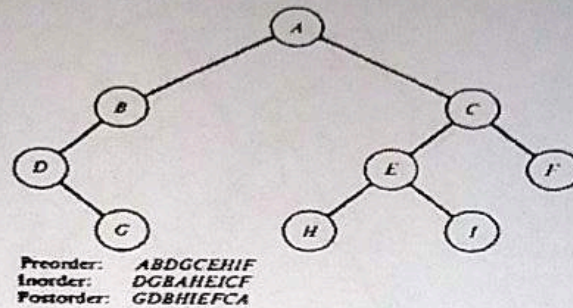


```
if(tree!=NULL)
{
    intrav(tree->left);
    printf("%d\n", tree->info);
    intrav(tree->right);
}
}
```

To traverse a nonempty binary tree in postorder;

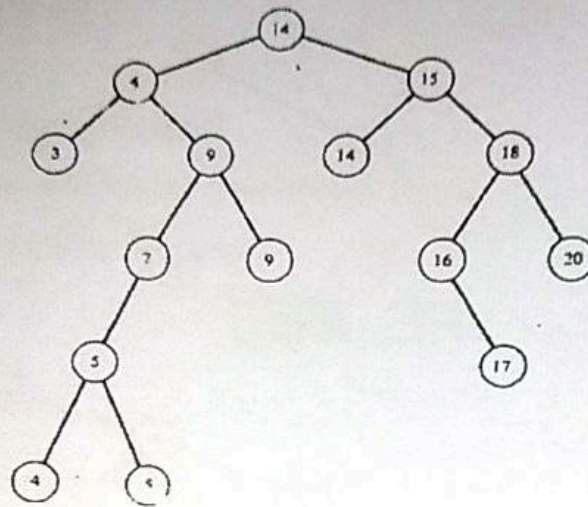
1. Traverse the left subtree in postorder.
2. Traverse the right subtree in postorder.
3. Visit the root.

```
void posttrav(NODEPTR tree)
{
    if(tree!=NULL)
    {
        posttrav(tree->left);
        posttrav(tree->right);
        printf("%d\n" tree->info);
    }
}
```

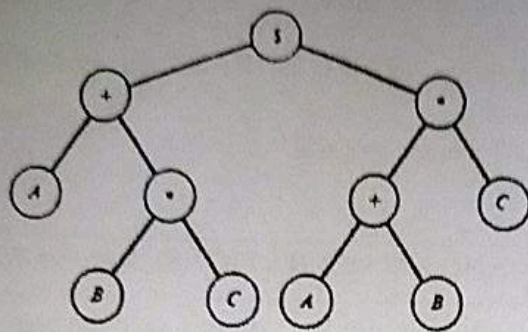
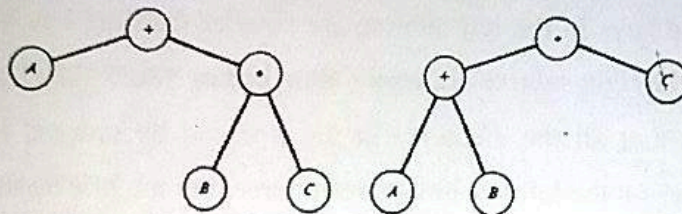
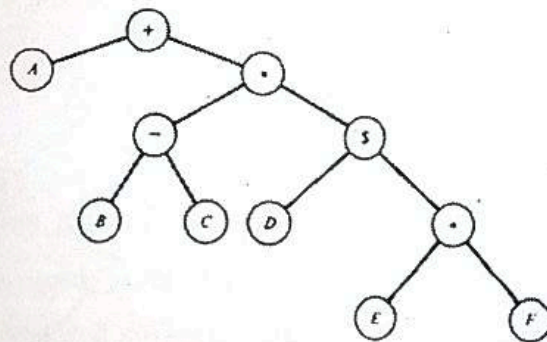
3. Sorting

- A binary tree that the property all elements in the left subtree of a node n are less than the contents of n , and all elements in the right subtree of n are greater than or equal to the contents of n .
- A binary tree that has a property is called a binary search tree. If a binary search tree's traversal in inorder (left, root, right) and the contents of each node are printed as the node is visited, the numbers are printed in ascending order.



4. Evaluation of Expression Trees

- As another application of binary tree, consider the following method of representing an expression containing operands and binary operators by a strictly binary tree.
- The root of the tree contains an operator that is to be applied to the results of evaluating the expressions represented by the left, and right subtrees.
- A node representing an operator is a nonleaf, whereas a node representing an operand is a leaf.
- A preorder traversal yields the prefix form of the expression.
- Similarly, traversing a binary expression tree in postorder places an operator after its two operands, so that a postorder traversal produces the postfix form of the expression.
- The postorder traversals of the binary trees yields the postfix forms.
- Since the root (operator) is visited after the nodes of the left subtree and before the nodes of the right subtree (the two operands), we might expect an inorder traversal to yield the infix form of the expression. Indeed, if the binary tree is traversed, the infix expression $A+B*C$ is obtained. However, a binary expression tree does not contain parentheses, since the operations are implied by the structure of the tree.
- Thus expressions whose infix form requires parentheses to override explicitly the conventional precedence rules cannot be retrieved by simple inorder traversals of the trees yield the expressions.

(d) $((A + B * C) * ((A + B) * C))$ (a) $A + B * C$ (b) $(A + B) * C$ (c) $A + (B - C) * DS(E * F)$

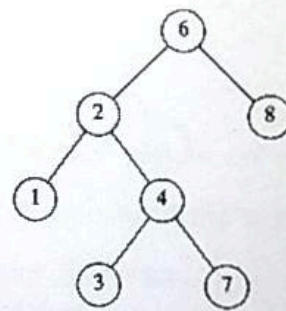
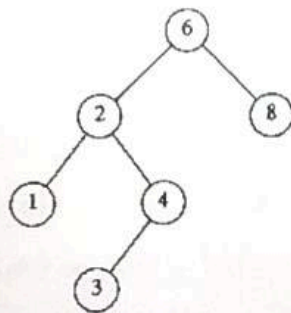
TYPES OF BINARY TREES

- There are several types of binary trees possible each with its own properties.
- Few important frequently used trees are listed as below.
 1. Expression tree
 2. Binary Search Tree
 3. Heap Tree
 4. Threaded Binary Tree

5. Huffmann Tree
6. Height balanced Tree (AVL Tree)
7. Weight balanced Tree
8. Decision Tree.

BINARY SEARCH TREES (BST) (AND ITS OPERATIONS)(6 Marks Nov 2015)

- An important application of binary trees is their use in searching.
- The property that makes a binary tree into a binary search tree is that for every node, X , in the tree, the values of all the keys in the left subtree are smaller than the key value in X , and the values of all the keys in the right subtree are larger than the key value in X .
- Notice that this implies that all the elements in the tree can be ordered in some consistent manner. In Figure, the tree on the left is a binary search tree, but the tree on the right is not.
- The tree on the right has a node with key 7 in the left subtree of a node with key 6 (which happens to be the root).



Two binary trees (only the left tree is a search tree)

```

typedef struct tree_node *tree_ptr;
struct tree_node
{
    element_type element;
    tree_ptr left;
    tree_ptr right;
  }
  
```



```
};  
typedef tree_ptr SEARCH_TREE;
```

Binary search tree declarations

Make_null

This operation is mainly for initialization. Some programmers prefer to initialize the first element as a one-node tree, but this implementation follows the recursive definition of trees more closely.

```
SEARCH_TREE make_null ( void )  
{  
    return NULL;  
}
```

Routine to make an empty tree

Find

This operation generally requires returning a pointer to the node in tree T that has key x , or $NULL$ if there is no such node. The structure of the tree makes this simple. If T is $NULL$, then return $NULL$. Otherwise, if the key stored at T is x , return T . Otherwise, make a recursive call on a subtree of T , either left or right, depending on the relationship of x to the key stored in T .

```
tree_ptr find( element_type x, SEARCH_TREE T )  
{  
    if( T == NULL )  
        return NULL;  
    if( x < T->element )  
        return( find( x, T->left ) );  
    else
```



```

    if( x > T->element )
        return( find( x, T->right ) );
    else
        return T;
}

```

Find operation for binary search trees

Find_min and find_max

These routines return the position of the smallest and largest elements in the tree, respectively. Although returning the exact values of these elements might seem more reasonable, this would be inconsistent with the *find* operation. It is important that similar-looking operations do similar things. To perform a *find_min*, start at the root and go left as long as there is a left child. The stopping point is the smallest element. The *find_max* routine is the same, except that branching is to the right child.

```

tree_ptr find_min( SEARCH_TREE T )
{
    if( T == NULL )
        return NULL;
    else
        if( T->left == NULL )
            return( T );
        else
            return( find_min ( T->left ) );
}

```

Recursive implementation of find_min for binary search trees

```

tree_ptr find_max( SEARCH_TREE T )
{

```



```

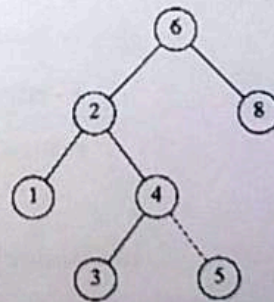
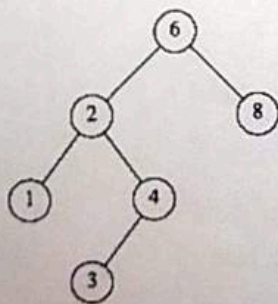
if( T != NULL )
    while( T->right != NULL )
        T = T->right;
    return T;
}

```

Nonrecursive implementation of find_max for binary search trees

Insert

- The insertion routine is conceptually simple.
- To insert x into tree T , proceed down the tree as you would with a *find*.
- If x is found, do nothing (or "update" something). Otherwise, insert x at the last spot on the path traversed. Figure shows what happens.
- To insert 5, we traverse the tree as though a *find* were occurring. At the node with key 4, go right, but there is no subtree, so 5 is not in the tree, and this is the correct spot.
- Duplicates can be handled by keeping an extra field in the node record indicating the frequency of occurrence.
- This adds some extra space to the entire tree, but is better than putting duplicates in the tree (which tends to make the tree very deep).
- Of course this strategy does not work if the key is only part of a larger record.
- If that is the case, then it is possible to keep all of the records that have the same key in an auxiliary data structure, such as a list or another search tree.



Binary search trees before and after inserting 5

- Following figure shows the code for the insertion routine.
- Since T points to the root of the tree, and the root changes on the first insertion, *insert* is written as a function that returns a pointer to the root of the new tree. Lines 8 and 10 recursively insert and attach x into the appropriate subtree.

```

tree_ptr insert( element_type x, SEARCH_TREE T )
{
    if( T == NULL )
    {
        T = (SEARCH_TREE) malloc ( sizeof (struct tree_node) );
        if( T == NULL )
            fatal_error("Out of space!!!");
        else
        {
            T->element = x;
            T->left = T->right = NULL;
        }
    }
    else
        if( x < T->element )
            T->left = insert( x, T->left );
        else
            if( x > T->element )
                T->right = insert( x, T->right );
            return T;
}

```

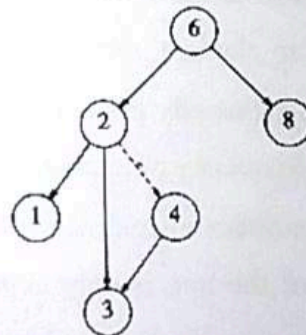
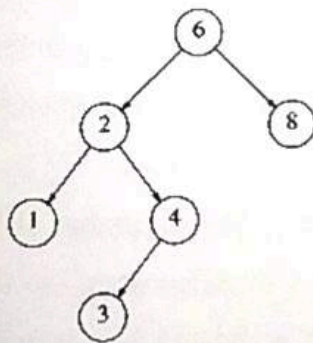
Insertion into a binary search tree

Delete

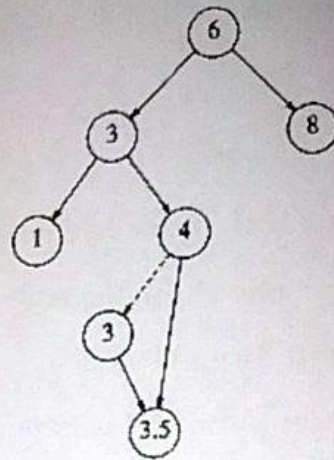
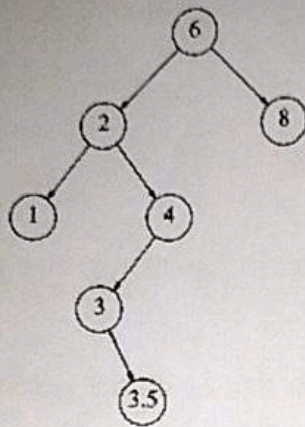
As is common with many data structures, the hardest operation is deletion.

Possibilities

- **node is a leaf**, it can be deleted immediately.
- **node has one child**, the node can be deleted after its parent adjusts a pointer to bypass the node.
- **node with two children.**
- The general strategy is to replace the key of this node with the smallest key of the **right subtree** (which is easily found) and recursively delete that node (which is now empty).
- Because the smallest node in the right subtree cannot have a left child, the second *delete* is an easy one. Figure shows an initial tree and the result of a deletion.
- The node to be deleted is the left child of the root; the key value is 2. It is replaced with the smallest key in its right subtree (3), and then that node is deleted as before.



Deletion of a node (4) with one child, before and after



Deletion of a node (2) with two children, before and after

- The code in Figure performs deletion. It is inefficient, because it makes two passes down the tree to find and delete the smallest node in the right subtree when this is appropriate.
- If the number of deletions is expected to be small, then a popular strategy to use is *lazy deletion*: When an element is to be deleted, it is left in the tree and merely *marked* as being deleted.
- This is especially popular if duplicate keys are present, because then the field that keeps count of the frequency of appearance can be decremented.
- If the number of real nodes in the tree is the same as the number of "deleted" nodes, then the depth of the tree is only expected to go up by a small constant, so there is a very small time penalty associated with lazy deletion. Also, if a deleted key is reinserted, the overhead of allocating a new cell is avoided.

```

tree_ptr delete( element_type x, SEARCH_TREE T )
{
    tree_ptr tmp_cell, child;
    if( T == NULL )
        error("Element not found");
    else

```



```
        if( x < T->element )
            T->left = delete( x, T->left );
        else
            if( x > T->element )
                T->right = delete( x, T->right );
            else
                if( T->left && T->right )
                {
                    tmp_cell = find_min( T->right );
                    T->element = tmp_cell->element;
                    T->right = delete( T->element, T->right );
                }
            else
            {
                tmp_cell = T;
                if( T->left == NULL )
                    child = T->right;
                if( T->right == NULL )
                    child = T->left;
                free( tmp_cell );
                return child;
            }
        return T;
    }
```

DELETION ROUTINE FOR BINARY SEARCH TREES

HEIGHT BALANCED

BINARY TREE

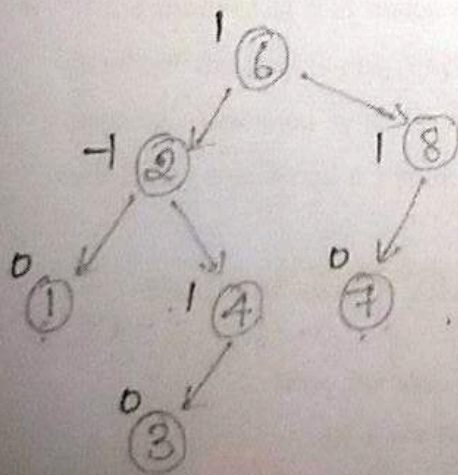
DEFENITION:-

A Binary Search Tree is said to be Height Balanced Binary Search Tree if all the nodes have a balance factor of 1, 0 (or) -1. That is

$$|bf| = |h_L - h_R| \leq 1$$

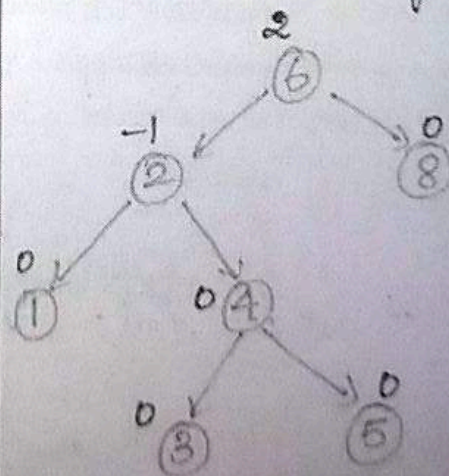
for every node in the tree.
For example:-

// It Satisfies the properties of Height Balanced Binary tree



(a) Height Balanced

// It does not satisfies the properties of Height Balanced Binary Tree.



(b) Height Unbalanced

BASIC OPERATIONS:-

- * Searching
- * Insertion
- * Deletion

⇒ These operations May not be with the minimum time but less than that of unbalanced Binary Search Tree.

⇒ In the Complete Binary Search Tree ,

- * Search time is minimum
- * Insertion and Deletion May not be with minimum time always.

FOLLOWING STEPS ARE INVOLVED:-

1. Insert node into a binary Search tree:

Insert the node into its proper position following the properties of binary Search tree.

this pointer manipulation is well known as AVL Rotation.

AVL Rotations

Case 1 :-

Unbalance occurred due to the insertion in the left sub-tree of the left child of the pivot node.

Manipulations in pointers take place:

- * Right Sub-tree (A_R) of left child (A) of pivot node (P) becomes the left sub-tree of P .

- * P becomes the right child of A .

- * Left Sub-tree (A_L) of A remains the same.

This case is called LEFT-TO-LEFT Insertion.

2. Compute the balance factors:-

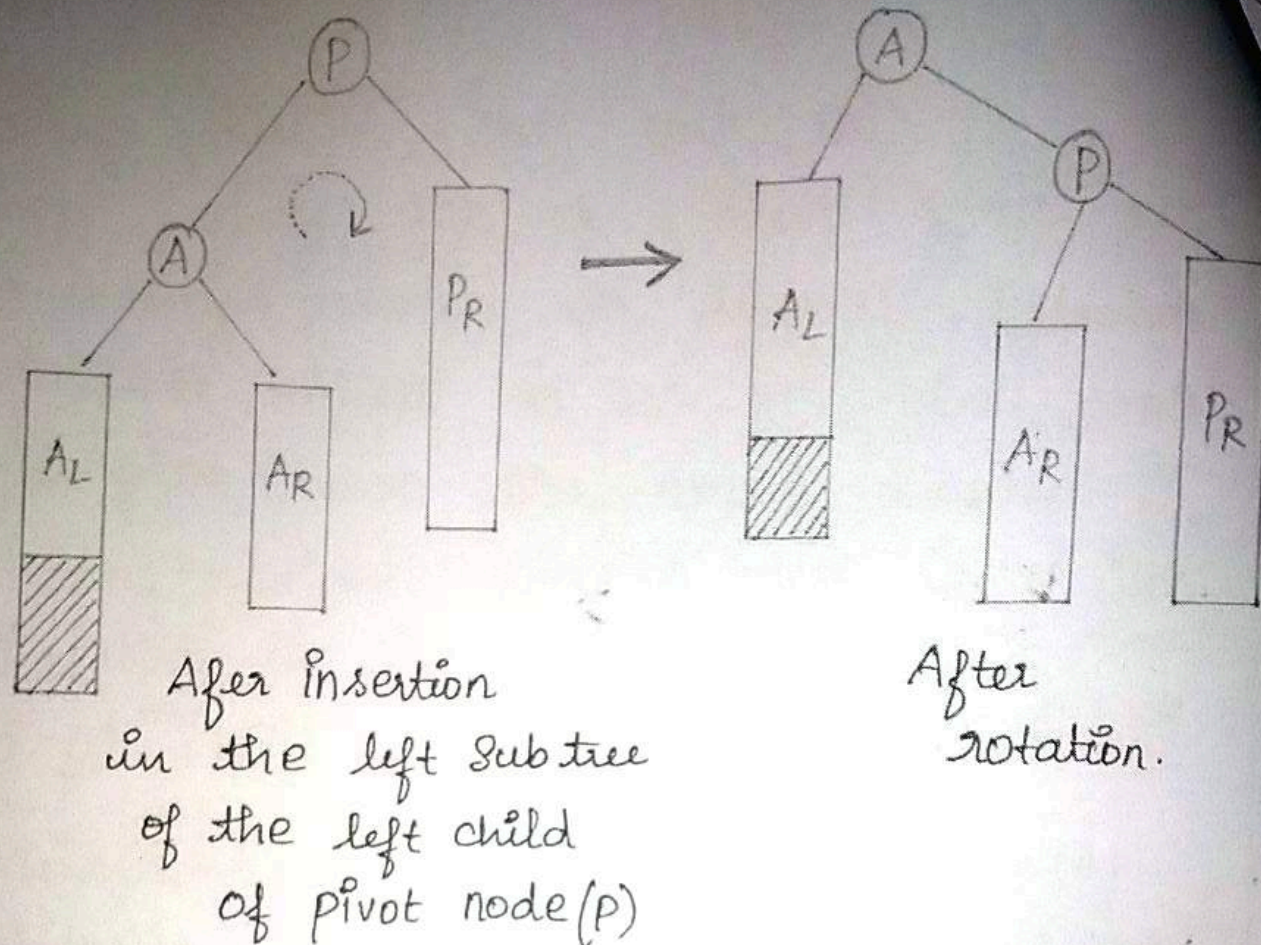
On the path starting from the root node to the node newly inserted, compute the balance factors of each node. It can be verified that change in balance factors will occur only in this path.

3. Decide the pivot node:-

On the path as traced in Step 2, determine whether the absolute value of any node's balance factor is switched from 1 to 2. If so, the tree becomes unbalanced. The node whose absolute value of balance factor is switched from 1 to 2, mark it as a special node called pivot node.

4. Balance the unbalance tree:-

It is necessary to manipulate pointers centred at the pivot node to bring the tree back into height balance.



Case 2 :-

Unbalance occurred due to the insertion in the right sub-tree of the right child of the pivot node.

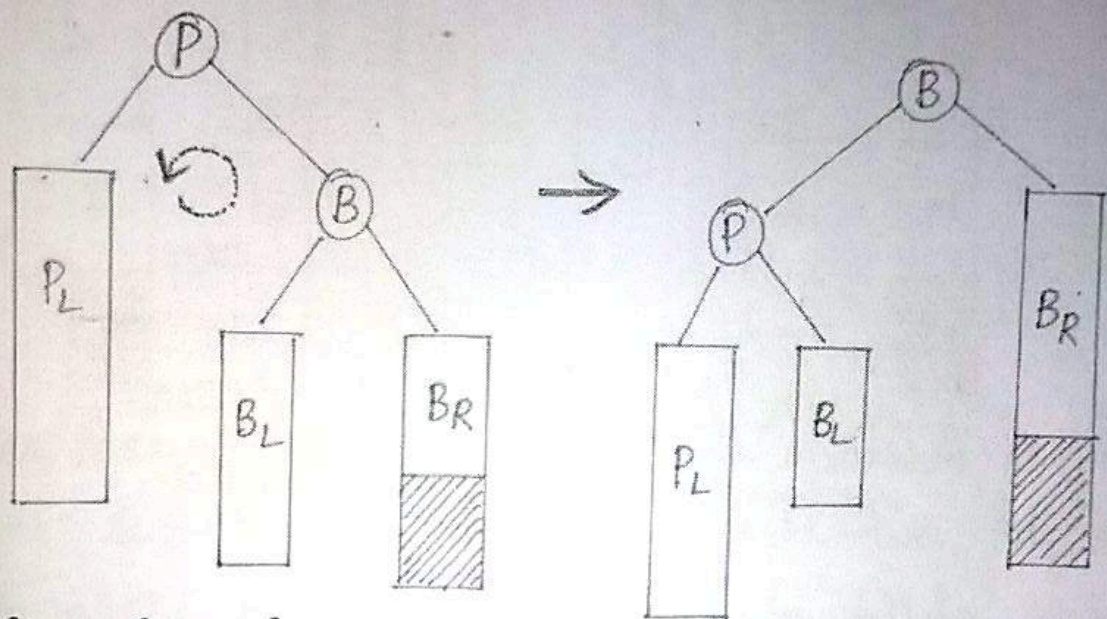
Manipulations in pointers take place:

- * Left sub-tree (B_L) of right child (B) of pivot node (P) becomes the right sub-tree of P .

- * P become the left child of B

* Right Sub-tree (BR) of B remains same

This is known as RIGHT-TO-RIGHT insertion.

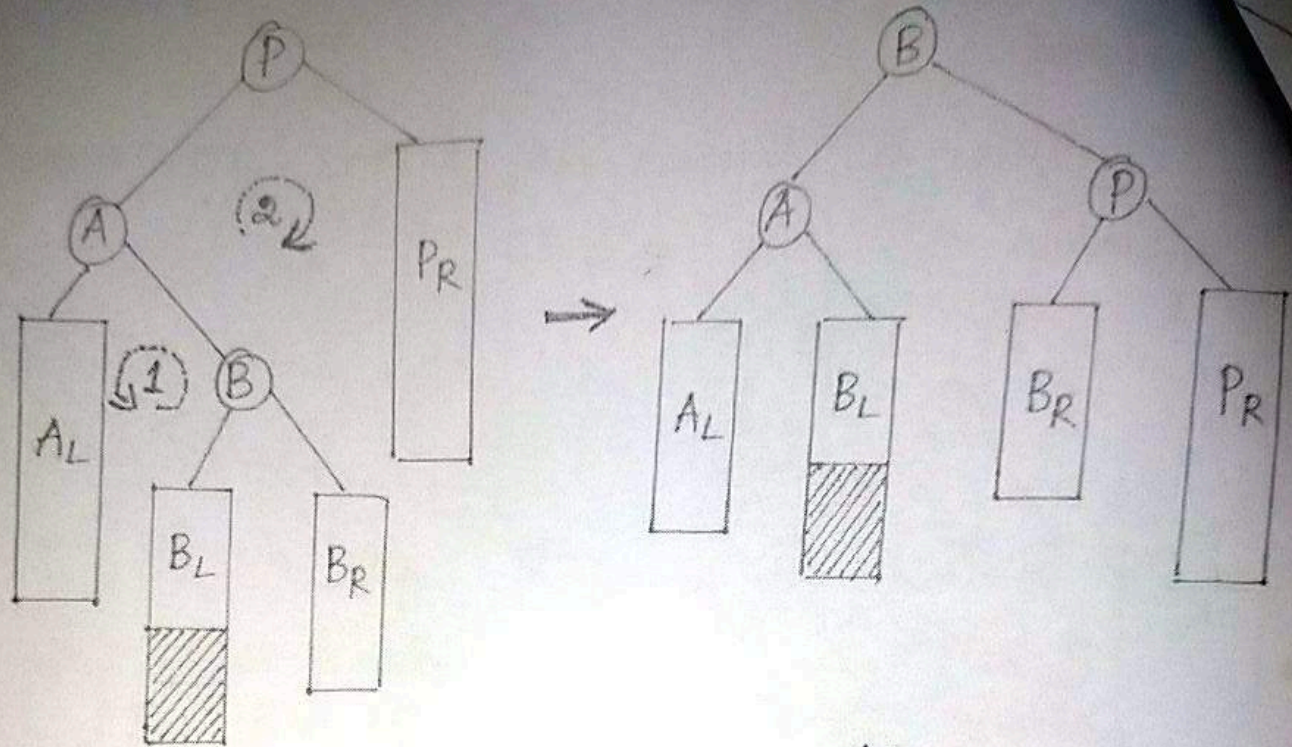


After insertion in the right sub-tree of the right child of pivot node (p)

After rotation

Case 3 :-

Unbalance occurred due to the insertion in the right sub-tree of the left child of the pivot node.



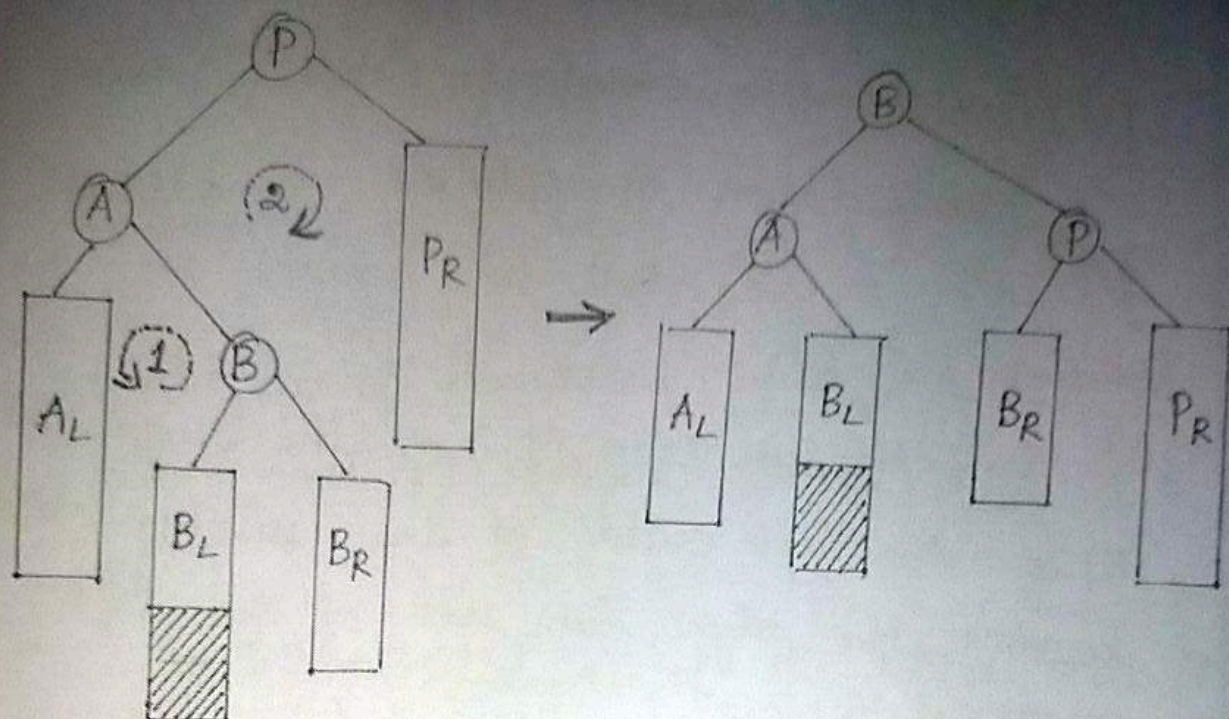
After insertion in the
right sub-tree of the
left child of pivot
node (p)

After
rotation

Case 4 :-

⇒ Unbalance occurred due to the
insertion in the left sub-tree of right
child of the pivot node.

⇒ This case is the reverse and
symmetric to Case 3.



After insertion in the
right sub-tree of the
left child of pivot
node (p)

After
rotation

Case 4 :-

⇒ Unbalance occurred due to the
insertion in the left sub-tree of right
child of the pivot node.

⇒ This case is the reverse and
symmetric to Case 3.

It involves two rotations for the
Manipulation in pointers.

Rotation 1

- * Left Sub-tree (B_L) of the right child (B) of the left child of the pivot node (P) becomes the right Sub-tree of the left child (A).

- * Left child (A) of the pivot node (P) becomes the left child of B.

Rotation 2 :-

- * Right Sub-tree (B_R) of the right child (B) of the left child (A) of the pivot node (P) becomes the left Sub-tree of P.

- * P becomes the right child of B.

This Case is known as

LEFT-TO-RIGHT Insertion.

In Involves two rotations for the manipulations of pointers:-

Rotation 1 :-

- * Right Sub-tree (B_R) of the left child (B) of the right child (A) of the pivot node (P) becomes the left sub-tree of A.

- * Right child (A) of the pivot node (P) becomes the right child of B.

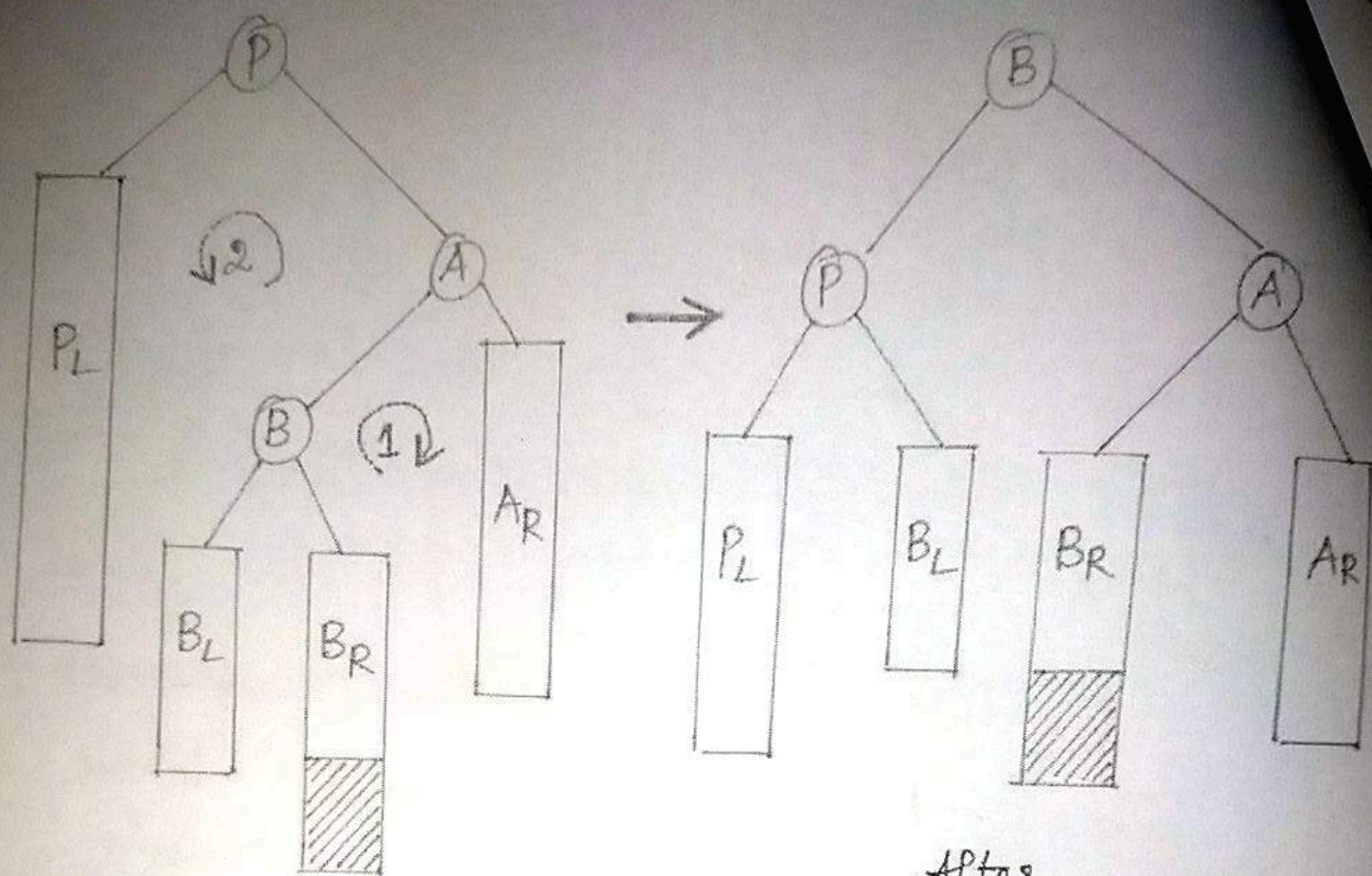
Rotation 2 :-

- * Left Sub-tree (B_L) of the right child (B) of the right child (A) of the pivot node (P) becomes the right Sub-tree of P.

- * P becomes the left child of B.

this case is known as

RIGHT - TO - LEFT insertion.



After insertion in the
left sub-tree of the
right child of pivot
node (P)

After
Rotation

IMPLEMENTATION

⇒ Implementation of a height balance tree is very straightforward.

⇒ Construct a height balanced binary tree, will follow the recursive approach of the insertion.

⇒ In order to make the understanding easy, we will modularize the whole procedure into several modules documented as below. We will assume the following node structure to maintain a height.

LCHILD	DATA	HEIGHT	RCHILD
--------	------	--------	--------

COMPUTE_HEIGHT(PTR):-

Algorithm:-

Input — pointer PTR to the tree whose height has to be calculated.

Output — Height of the tree.

Data Structure — Linked Structure of height balanced binary tree.

Steps :-

If (PTR = NULL) then

{

height = 0

Return (height)

}

Else

{

lptr = PTR.LCHILD

rptr = PTR.RCHILD

}

$h_L = \text{COMPUTE_HEIGHT}(lptr)$

$h_R = \text{COMPUTE_HEIGHT}(rptr)$

If ($h_L \leq h_R$) then

{

height = 1 + h_R

}

else

{

height = 1 + h_L

}

IMPLEMENTATION

⇒ Implementation of a height balance tree is very straightforward.

⇒ Construct a height balanced binary tree, will follow the recursive approach of the insertion.

⇒ In order to make the understanding easy, we will modularize the whole procedure into several modules documented as below. We will assume the following node structure to maintain a height.

LCHILD	DATA	HEIGHT	RCHILD
--------	------	--------	--------

COMPUTE_HEIGHT (PTR):-

Algorithm:-

Input — pointer PTR to the tree whose height has to be calculated.

Output — Height of the tree.

ROTATION_RIGHT_TO_RIGHT (Pptr)

Algorithm:-

Input — pointer pptr to the pivot node

Output — AVL rotation corresponding to the unbalance due to insertion at the right sub-tree of right child of Pptr.

Steps:-

Bptr = pptr.RCHILD

Pptr.RCHILD = Bptr.LCHILD

Bptr.LCHILD = Pptr

Pptr.HEIGHT = COMPUTE_HEIGHT(Pptr)

Bptr.HEIGHT = COMPUTE_HEIGHT(Bptr)

Pptr = Bptr

Stop.

ROTATION_LEFT_TO_RIGHT (Pptr)

Algorithm:-

Input — pointer pptr to the pivot node.

Output — AVL rotation corresponding to the unbalance due to insertion in the right sub-tree of left child of Pptr.

Endif

Return (-height)

Endif

Stop

ROTATION_ LEFT_TO_LEFT (Pptr)

Algorithm:-

Input - pointer Pptr to the pivot node.

Output - AVL rotation corresponding to the unbalance due to insertion in the left sub-tree of left child of Pptr.

Steps:-

Aptr = Pptr.LCHILD

Pptr.LCHILD = Aptr.RCHILD

Aptr.RCHILD = Pptr

Pptr.HEIGHT = COMPUTE_HEIGHT (Pptr)

Aptr.HEIGHT = COMPUTE_HEIGHT (Aptr)

Pptr = Aptr

Stop

ROTATION_RIGHT_TO_RIGHT (Pptr)

Algorithm:-

Input — pointer pptr to the pivot node

Output — AVL rotation corresponding to the unbalance due to insertion at the right sub-tree of right child of Pptr.

Steps:-

Bptr = pptr.RCHILD

Pptr.RCHILD = Bptr.LCHILD

Bptr.LCHILD = pptr

Pptr.HEIGHT = COMPUTE_HEIGHT(Pptr)

Bptr.HEIGHT = COMPUTE_HEIGHT(Bptr)

Pptr = Bptr

Stop.

ROTATION_LEFT_TO_RIGHT (Pptr)

Algorithm:-

Input — pointer pptr to the pivot node.

Output — AVL rotation corresponding to the unbalance due to insertion in the right sub-tree of left child of Pptr.

Steps:-

$Aptr = Pptr \cdot LCHILD$

ROTATION_RIGHT_TO_RIGHT($Aptr$)

ROTATION_LEFT_TO_LEFT($Pptr$)

Stop.

ROTATION_RIGHT_TO_LEFT($Pptr$)

Algorithm:-

Input — pointer $Pptr$ to the pivot node

Output — AVL rotations corresponding to the unbalance due to insertion in the left sub-tree of right child of $Pptr$.

Steps:-

$Aptr = Pptr \cdot RCHILD$

ROTATION_LEFT_TO_LEFT($Aptr$)

ROTATION_RIGHT_TO_RIGHT($Pptr$)

Stop

INSERT_HEIGHT_BALANCED_TREE (ROOT, Nptr)

Algorithm:-

Input — Nptr is the pointer to a newly available node which has to be inserted.

Output — Nptr is inserted into the tree if its data component does not exist already.

Data Structure — Linked structure of height balanced binary tree, ROOT is the pointer to the root node of the tree
(ROOT = NULL $\Rightarrow$ tree is initially empty)

Steps:-

PTR = ROOT

If (PTR = NULL) then
{

PTR = Nptr

PTR.HEIGHT = 1

return()

}

else

{

If (Npta . DATA < pta . DATA) then

{

INSERT_HEIGHT_BALANCED_TREE (pta . LCHILD,
Npta

Lpta = pta . LCHILD

Rpta = pta . RCHILD

If (Rpta = NULL) then

{

h_R = 0

}

else

{

h_R = Rpta . HEIGHT

h_L = Lpta . HEIGHT

b_f = h_L - h_R

If (b_f = 2) then

{

If (Npta . DATA < Lpta . DATA) then

{

ROTATION_LEFT_TO_RIGHT (pta)

}

else

(B TREE)

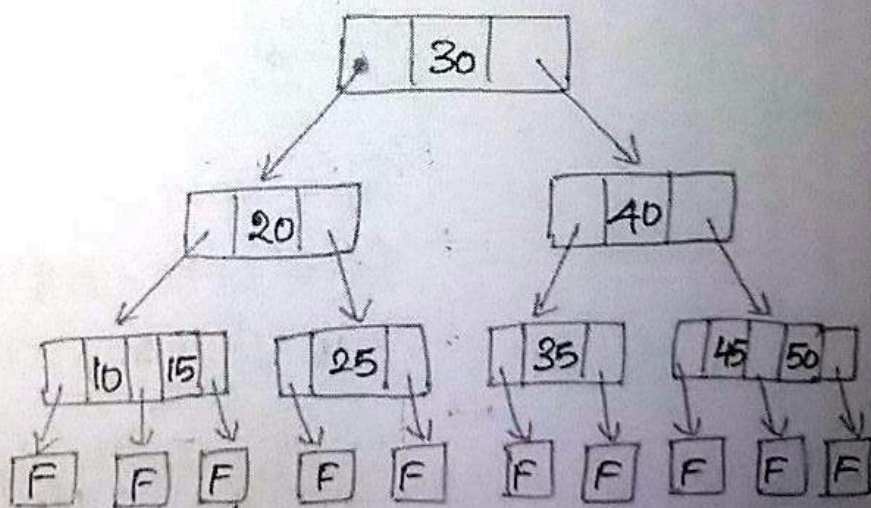
DEFINITION:-

A B tree T of order m is an m -way search tree, that is, either it is empty or it satisfies the following properties.

- * The root node has at least 2 children
- * All nodes other than the root node have at least $\lceil m/2 \rceil$ children.
- * All failure nodes are at the same level.

OPERATIONS:-

- * SEARCHING
- * INSERTING
- * DELETING
- * DISPLAYING



F → Failure node.

SEARCHING:-

Searching for a key value in a B tree is almost same as that of Searching a Key Value in a binary Search tree.

To perform Searching:-

Over it for some i such that

$$K_i \leq x < K_{i+1}$$

Assume

$$K_0 = -\infty \text{ \& } K_{n+1} = +\infty$$

Two Cases:-

Case 1:- $x = K_i$, Match is occur, search is successful and complete.

Case 2:- $K_i < x < K_{i+1}$

$\Rightarrow$ No match is found

$\Rightarrow$ Search has to propagated in sub tree whose pointer is stored in p_i

$\Rightarrow p_i = \text{NULL}$

then search is unsuccessful.

Algorithm:-

Input — AB tree of order m

Pointer to the root node is BTR

x is the key value.

Output — If Key Value is found, $\text{flag} = 1$,
Address of the node & Index of the Key
value found else $\text{flag} = 0$.

Data Structures — Array of pointers $\text{PTRS}[m]$
Array of Keyvalues $\text{KEYS}[m+1]$

Steps:

```
Ptr = BTR, KEYS[0] =  $-\infty$ , flag = FALSE
while (Ptr  $\neq$  NULL) A(flag = FALSE) do
  for (i = 0 to (m-1)) do
    PTRS[i] = Ptr  $\rightarrow$  Pi
  Endfor
  i = 1, number = 0
  while (Ptr, Ki  $\neq$  NULL) do
    KEYS[i] = Ptr  $\rightarrow$  Ki
    number = number + 1, i = i + 1
  Endwhile
  KEYS[number + 1] =  $+\infty$ , i = 0
  while (X > KEYS[i + 1])
    i = i + 1
  Endwhile
  If X = KEYS[i + 1]
    flag = TRUE
  Return (flag, Ptr  $\rightarrow$  Pi, i + 1)
```


Exit

Else

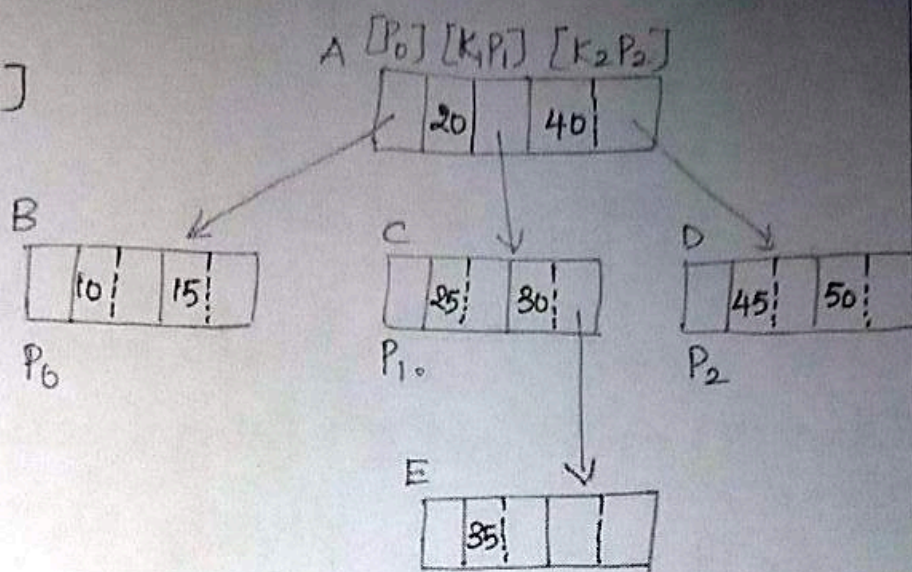
$Ptr = PTRS[P]$

Endif

Endwhile

Stop

Eg:



INSERTING:-

One can include a key value into a B tree. (or) Inserting root node and its child nodes.

Algorithm:-

STEPS:-

Input — Void insert (node ** tree, int val)

{

node * temp = NULL;

if (!(*tree))

{

temp = (node *) malloc(sizeof(node));

temp->left = temp->right = NULL;

temp->data = val;

*tree = temp;

INSERTION:

One can include a key value into a B-tree.

MAJOR 2 STEPS:

① Search the B tree to check if x already exists.

② If x already exists then no insertion, otherwise select the right node where x might be placed.

③ Place x into right node, obviously satisfying the properties of B tree of order m .

ALGORITHM:

I/P: A pointer BTR to the root node of B tree; x is the Key Value that has to be inserted.

O/P: Successful insertion of x into B tree if x does not already exist, else a message.

DATA STRUCTURES.

(1) An array $TEMP[m+1]$ to store the tuples $(k_i, p_i), 0 \leq i \leq m$ of a node.

(2) A stack $PATH$ to hold the pointers of nodes.

1. $k = x, p = NULL, TEMP[0].k = -\infty$
2. $[ptr, flag, path] = \text{Search Node}(BTR, x)$
3. If $(flag = 1)$ then
4. Print ("Key value exists in Btree")
5. Exit
6. End If
7. While $(ptr \neq NULL)$ do
8. For $i = 0$ to $(m-1)$ do
9. $TEMP[i].p = ptr \rightarrow p_i$
10. End For
11. For $i = 1$ to $(m-1)$ do
12. $TEMP[i].k = ptr.k_i$
13. End For
14. $TEMP[m].k = NULL$
15. $TEMP.Insert(k, p)$
16. $number = 0, i = 1$
17. While $(TEMP[i].k \neq NULL \text{ and } i \leq m)$ do


```
18.      number = number + 1,  $i = i + 1$ 
19.      End while
20.  If (number  $\leq (m-1)$ ) then
21.      For  $i = 0$  to number do
22.          ptr  $\rightarrow P_i = TEMP[i].P$ 
23.      End for
24.      For  $i = 1$  to number do
25.          ptr  $\rightarrow k_i = TEMP[i].k$ 
26.      End For
27.      For  $i = (number + 1)$  to  $(m-1)$  do
28.          ptr  $\rightarrow P_i = NULL,$ 
29.          ptr  $\rightarrow k_i = NULL$ 
29.      End For
30.  Exit
```


7. DELETION.

* Deletion of a key value from a B tree may take place in several situations.

* Each situation has a unique rule for managing a deletion.

* Whatever the situation, the deletion of a node must ensure the properties of a B tree.

2 properties are

① The root node must have at least 1 key value.

② All other nodes (except the root node) must have at least $\lceil m/2 \rceil - 1$ Key Values.

Cases for deletion:

① DELETION OF A KEY VALUE FROM A LEAF NODE

Removal of a key value leads to the number of keys $\geq \lceil m/2 \rceil - 1$

Removal of key value leads to the number of keys $< \lceil m/2 \rceil - 1$

② DELETION OF A KEY VALUE FROM A

NON-LEAF NODE

ALGORITHM:

I/P: BTR, pointer to root node of Btree.
K, the key value which has to be removed.

O/P: B tree after the removal of the key K if it exists else a message.

Data Structures:

A stack PATH to store the path from the root node to any node in Btree.

This object is returned by SEARCH_NODE (BTR, K) method.

Steps:

1. Search for the key value K:

⇒ Perform SEARCH_NODE (BTR, K) to see whether key value K is in Btree BTR or not.

⇒ If K is not in Btree, print the message and exit:

⇒ If K exists, let it be in the node P as K_l (in lth location)

DELETION.

* Deletion of a key value from a B tree may take place in several situations.

* Each situation has a unique rule for managing a deletion.

* Whatever the situation, the deletion of a node must ensure the properties of a B tree.

2 properties are

① The root node must have at least 1 key value.

② All other nodes (except the root node) must have at least $\lceil m/2 \rceil - 1$ key values.

Cases for deletion:

① DELETION OF A KEY VALUE FROM A LEAF NODE

Removal of a key value leads to the number of keys $\geq \lceil m/2 \rceil - 1$

Removal of key value leads to the number of keys $< \lceil m/2 \rceil - 1$

② DELETION OF A KEY VALUE FROM A NON - LEAF NODE

⇒ Let n be the number of key values in P (after removal of K_i).

⇒ While $(n < \text{MIN})$ AND $(P \neq \text{BTR})$ do

① $\text{MIN} = \lfloor m/2 \rfloor - 1$, the minimum number of key values in a node.

② Various cases, when on removal of key values from P results in less than the MIN no. of key values and P is not root node.
(note : root node may contain even $< \text{MIN}$ key values).

(i) Let z be parent of P and X and $Y \rightarrow$ left and right siblings of P respectively.

(ii) CASE 1:

Right siblings (Y) exists
and

no. of key values in $X > \text{MIN}$

Perform Move left
Exit.

⇒ Let n be the number of key values in P (after removal of K_x).

⇒ While $(n < \text{MIN})$ AND $(P \neq \text{BTR})$ do

① $\text{MIN} = \lceil m/2 \rceil - 1$, the minimum number of key values in a node.

② Various cases, when on removal of key values from P results in less than the MIN no. of key values and P is not root node.
(note : root node may contain even $< \text{MIN}$ key values).

(i) Let z be parent of P and x and $y \rightarrow$ left and right siblings of P respectively.

(ii) CASE 1 :

Right siblings (y) exists
and

no. of key values in $x > \text{MIN}$

Perform Move left
Exit.

(iii) CASE 2:

Left sibling (X) exists

and

no. of key values in X > MIN

Perform Move 'right'

Exit

(v) CASE 4

(iv) CASE 3:

LEFT SIBLING (X)

Right sibling (Y) exists

but

no. of key values in Y is MIN

Perform 'Combine' with
right sibling

Let n become the number of key
values in parent of P ^{also X} also Y due
to this 'Combine'.

⇒ End of while

⇒ If (n=0), then BTR is the pointer
P₀ in P. Return the empty node.

4. Stop

TRAVERSAL OF B TREE

DEFINITION:-

To display all the key values on a B tree means to traverse all the nodes in the B tree in tree inorder fashion.

Algorithm:-

Input — AB tree with elements and pointer to the root node BTR.

Output — Traverse all key values in ascending order.

Data Structure — Linked structure of Btree with pointer to the root node is BTR.

Steps:-

```
ptr = BTR
If (ptr ≠ NULL) then
  Traverse Btree (ptr → P0)
  For (i = 1 to (m-1)) do
    Print (ptr → Ki)
  Traverse Btree (ptr → Pi)
Endfor
Endif
Stop
```


UPPER BOUND OF A B-TREE :

Q B tree is always a height balanced tree

Q Degree of B tree of order m is m ,
(i.e) maximum number of branches
that can emanate from a node
is m .

Q In a B tree of order m and
height h ;

Q MAXIMUM NO. OF NODES POSSIBLE :

$$\sum_{i=0}^{h-1} m^i = \frac{m^h - 1}{m - 1}$$

Q Maximum no. of Key values = $(m-1)$
(where, B tree of order m)

Q Maximum no. of Key values that
is possible : (for order m)

$$\frac{m^h - 1}{m - 1} \times (m - 1) = m^h - 1$$

LOWER BOUND OF A B-TREE :

* Root node contains atleast 2 children.

* All nodes other than root node can
have atleast $\lceil m/2 \rceil$ children.

Minimum no. of Key values in root

* Minimum no of key values in any node other than root node is $\lceil m/2 \rceil - 1$

TO CALCULATE MIN. NO. OF KEY VALUE OF ORDER m :

Suppose all the failure nodes are at level l . Then, min. no. of nodes possible at various levels is given below:

LEVEL	MINIMUM NUMBER OF NODES	REMARK
0	1	Root node
1	2	Root node has atleast 2 children.
2	$2 \times \lceil m/2 \rceil$	Each node other than the root node.
3	$2 \times \lceil m/2 \rceil \times \lceil m/2 \rceil$	$\vdots$
$\vdots$	$\vdots$	$\vdots$
$l-1$	$2 \times \lceil m/2 \rceil^{l-1}$	Minimum no - of nodes in last level.

$\Rightarrow$ The above nodes so counted are non-failure nodes.

$\Rightarrow$ Now, suppose there are atleast N no. of key values $k_1, k_2, \dots, k_N$, where $k_i < k_{i+1}$ for $1 \leq i \leq n$.

Then, No. of failure nodes =

$$\begin{aligned}
 N+1 &= \text{No. of failure nodes in B tree} \\
 &= \text{No. of nodes in level } l. \\
 &\geq 2 \times [m/2]^{l-1+1} \\
 &\geq 2 \times [m/2]^l \\
 N &\geq 2 \times [m/2]^l - 1
 \end{aligned}$$

Therefore,

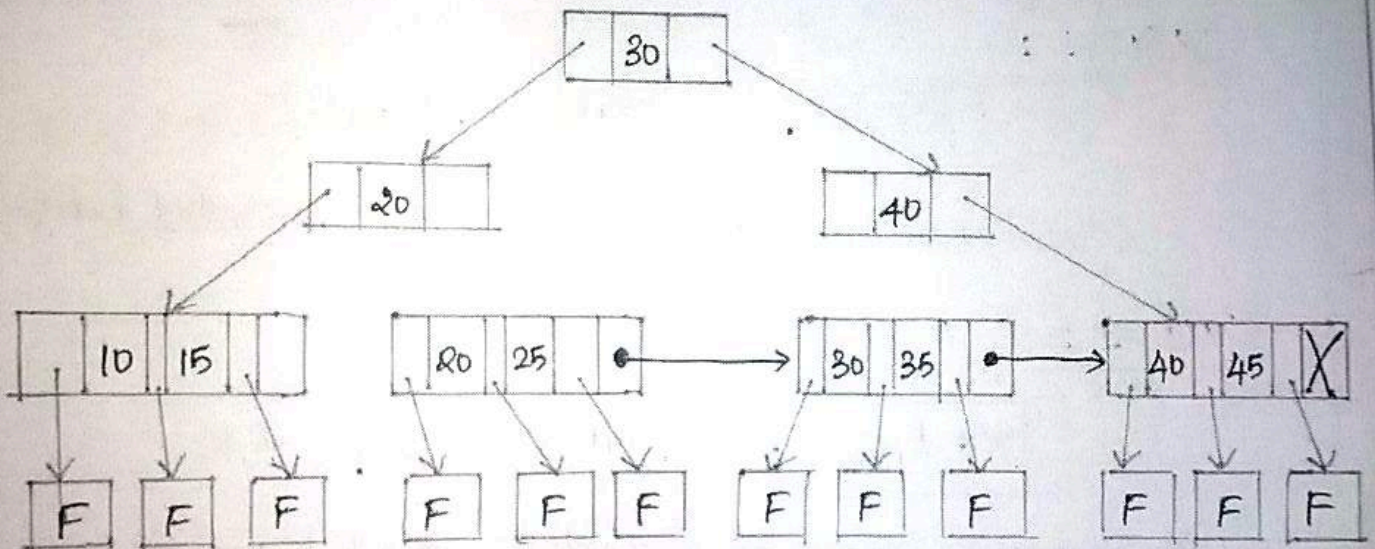
No. OF NODES $\Rightarrow$ B tree of order m may contain atleast $2 \times [m/2]^l - 1$ no. of nodes.

— x — x —

B+ TREE INDEXING

DEFINITION:-

A slight modification of indexing in B tree is known as B+ tree indexing. The primary distinction between the two approaches is that, a B+ tree allows redundant storage of key values.



F = failure node.

OPERATIONS:-

- * SEARCHING
- * INSERTING
- * DELETING

SEARCHING:-

⇒ Searching a Key Value in a B+ tree

⇒ Find the Key Value K , we should examine the root node and look for the smallest key value which is greater than K .

* If Key is K_i , K_i will obey the following three cases:-

Case 1 :-

$$\boxed{K_i = K_1}, \text{ that is, } K < K_1$$

We have to go to the node pointed by P_0 .

Case 2 :-

$$\boxed{K_i = K_{m-1}}, \text{ that is, } K_{m-1} < K$$

We have to go to the node pointed by the pointer P_{m-1} .

Case 3 :-

$$\boxed{K_i < K < K_{i+1}}, \text{ where } 1 < i < m-1$$

We have to go to the node pointed by the pointer P_{i+1} .

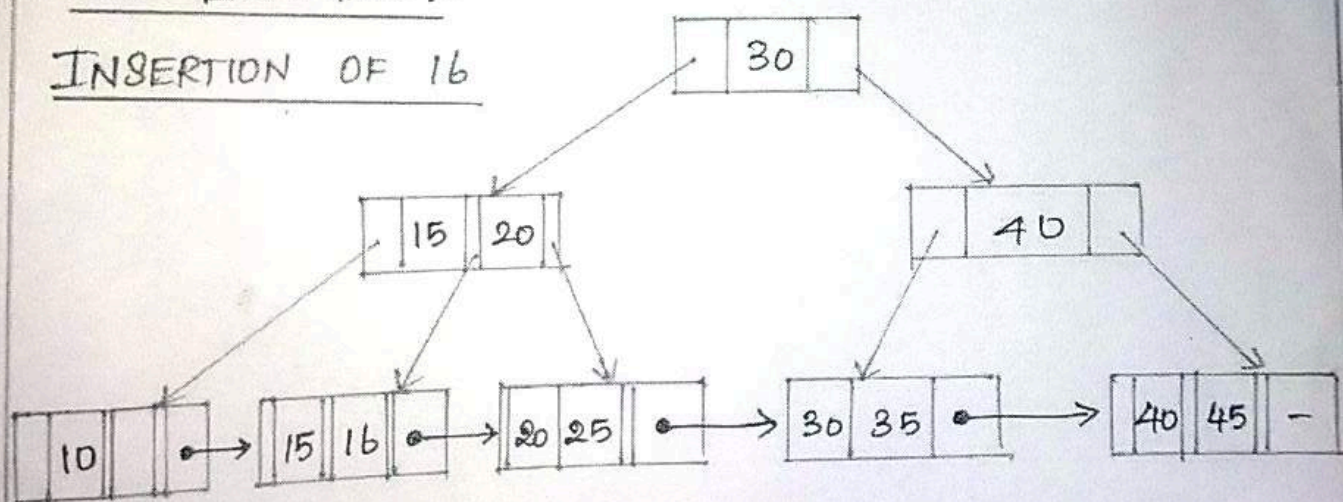
INSERTING :-

⇒ In Order to insert a key value, first search for the key value using the same technique as stated above to decide the leaf node in which the key value would appear.

⇒ If key value already exists, no insertion occurs.

⇒ Insert the value in leaf node in an ordered fashion, follow the splitting of node
FOR EXAMPLE:-

INSERTION OF 16



DELETING:-

⇒ To delete a key value, perform the search for K that has to be deleted.

⇒ The key value will be removed in the same style as that of B tree.